INNOMATICS RESEARCH LABS

Table Of Contents

Table Of Contents
Why Object-Oriented Programming?
- Object-Oriented Approach – Customer as an Object
- Why This Approach is Better

Introduction to Object-Oriented Programming
- Real-World Parallel: The Bakery Example
- How OOP Works
- Key Benefits of OOP

What is a Class and Object?
- 1. Class: The Blueprint
- 2. Object: The Instance
- 3. Example: Defining and Using a Class
- 4. Visual Understanding
- 5. Why This Separation Matters
- 6. Practice Task

The __init__ Method and Instance Variables
- 1. What is the __init__ Method?
- 2. What are Instance Variables?
- 3. Example: Using __init__ and Instance Variables
- 4. Visual Summary
- 5. Common Mistakes to Avoid

Understanding Methods: Instance Methods, Class Methods, and Static Methods
- 1. Instance Methods
- 2. Class Methods
- 3. Static Methods
- Practice Task

Encapsulation
1. What is Encapsulation?
- 2. Encapsulation in Python
- 3. Example: Public, Protected, and Private Variables
- 4. Getter and Setter Methods
- 5. Why Use Encapsulation?

Inheritance
- 1. What is Inheritance?
- 2. Why Use Inheritance?
- 3. Example: Inheritance in Action

- 4. Types of Inheritance in Python
  - Single Inheritance
  - Multiple Inheritance
  - Multilevel Inheritance
  - Hierarchical Inheritance
  - Hybrid Inheritance
  - Summary
- Method Overriding in Python
  - What is Method Overriding?
  - Why Override Methods in Child Classes?
  - Using super() to Call Parent Method
  - MRO Behavior in Multiple Inheritance
- Constructor Overriding in Python
  - What is Constructor Overriding?
  - How It Affects Inheritance
  - Calling Parent Constructor Using super()
  - Why Use Constructor Overriding
- Polymorphism in Python
  - What is Polymorphism?
  - Why Use Polymorphism?
  - Method Overriding (Real Polymorphism in Python)
  - Polymorphism with Functions
  - Important Points
- Abstraction
  - What is Abstraction?
  - Why Use Abstraction?
  - How Abstraction Works in Python?
    - Syntax:
    - Key Points:

Why Object-Oriented Programming?

Imagine you're running a small bakery.

In the **procedural style**, you manage everything through individual steps:

- You keep separate notebooks for customer names, orders, and payments.
- Every time someone places an order, you manually check all three books to find their details, write down their new order, and update their payment record.
- As the number of customers grows, things start to get disorganized. You may mix up orders, forget to update payments, or misplace customer information.

Now imagine switching to an **object-oriented approach**:

- For every customer, you create a **customer file** that contains all their information: name, orders, and payments — all in one place.
- When a customer places a new order, you simply open their file and update it.
- Everything related to that customer stays together, making the system easier to scale and less error-prone.

This is the core difference between procedural and object-oriented programming:

Procedural Programming	Object-Oriented Programming
Separate pieces of data and functions	Combines data and related functions in one unit
Harder to manage as size increases	Easier to scale, modify, and reuse
Manual coordination between multiple parts	Each object manages its own behavior and data

Procedural Approach – Functions Only

Let’s implement the bakery example using functions.

```python
# Procedural Approach

# Using separate functions to manage customer data and orders
def record_customer(name, phone):
    print(f"Customer Name: {name}")
    print(f"Phone: {phone}")

def add_order(item, quantity):
    print(f"Order Item: {item}")
    print(f"Quantity: {quantity}")

def process_payment(amount):
    print(f"Amount Paid: ₹{amount}")

# Using the system
record_customer("Anjali", "9876543210")
add_order("Chocolate Cake", 2)
process_payment(500)
```

This works for small cases, but what if:

- The same customer places multiple orders?
- You need to track delivery address, feedback, or discounts?
- The bakery grows and now you need to manage hundreds of such orders?

Handling it all through separate functions becomes difficult to manage and debug.

Object-Oriented Approach – Customer as an Object

Let's refactor the same logic using classes and objects.

```python
# Object-Oriented Approach

class Customer:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.orders = []
        self.total_payment = 0
```

```
    def add_order(self, item, quantity, price_per_item):
        self.orders.append((item, quantity, price_per_item))
        self.total_payment += quantity * price_per_item

    def show_summary(self):
        print(f"Customer: {self.name} ({self.phone})")
        print("Orders:")
        for item, qty, price in self.orders:
            print(f"- {item} x {qty} @ ₹{price} each")
        print(f"Total Payment: ₹{self.total_payment}")

# Creating a customer and managing their orders
customer1 = Customer("Deva", "9876543210")
customer1.add_order("Chocolate Cake", 2, 250)
customer1.add_order("Pastry", 4, 60)
customer1.show_summary()
```

Why This Approach is Better

- The customer's **data** and **behavior** are bundled together inside a single object.
- You can create multiple customer objects, each with their own orders and payment history.
- The program is cleaner, modular, and easier to maintain or expand (e.g., adding delivery status or feedback features).

Introduction to Object-Oriented Programming

Object-Oriented Programming, or OOP, is a programming paradigm that focuses on **objects** — entities that combine **data** and **behavior**. Instead of writing code that performs actions on raw data (as in procedural programming), OOP encourages organizing the program around **real-world entities** and their interactions.

This approach allows you to build systems that are:

- Easier to understand
- More modular and reusable
- Simpler to maintain and extend

Real-World Parallel: The Bakery Example

In our earlier bakery example:

- **Procedural Approach**: You used separate functions to record customer details, manage orders, and handle payments. This led to fragmented code where data and related logic were handled separately.
- **Object-Oriented Approach**: You grouped all customer-related data and behavior inside a `Customer` class. Each customer became an object with its own attributes (name, phone, orders) and methods (`add_order()`, `show_summary()`).

How OOP Works

At the heart of OOP are two fundamental concepts:

- **Class**: A blueprint or template for creating objects. It defines what data the object holds and what operations it can perform.
- **Object**: A specific instance of a class with actual values. You can create multiple objects from the same class.

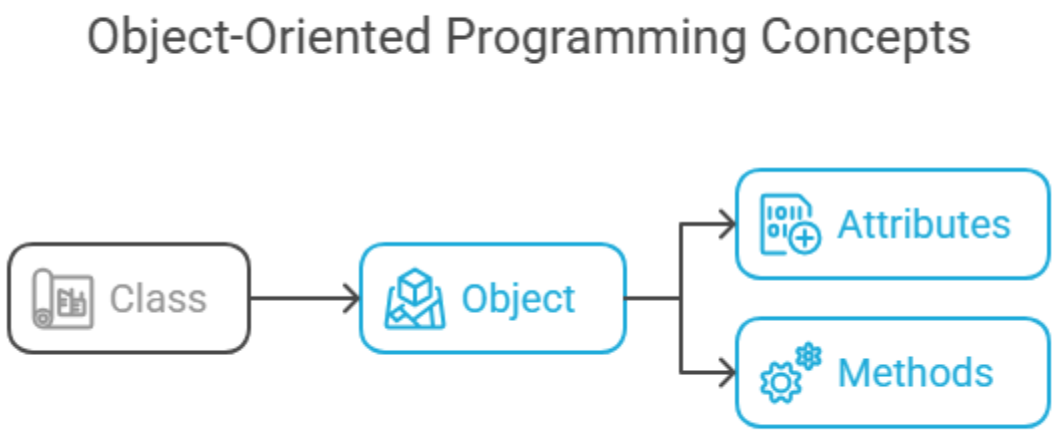
Key Benefits of OOP

1. **Reusability**
   Write a class once and create many objects from it.
2. **Encapsulation**
   Keep data and behavior together, hiding internal details from outside access.
3. **Scalability**
   Easy to add new features by extending classes or creating new ones.
4. **Maintainability**
   Code changes are localized, reducing the risk of breaking other parts of the program.

Object-Oriented Programming enables you to design programs that:

- Reflect real-world entities and relationships
- Keep related data and behavior in one place
- Scale better as programs grow in size and complexity

What is a Class and Object?



1. Class: The Blueprint

A **class** is a blueprint or template used to create objects. It defines the **structure** (what data the object will hold) and **behavior** (what actions it can perform), but it does **not** hold any actual data by itself. Think of a **class as a recipe**. The recipe defines ingredients and steps, but the actual cake isn't made until you follow the recipe.

Syntax of a Class in Python:

```
class ClassName:
    def __init__(self):
        # attributes
        pass

    def method_name(self):
        # behavior
        pass
```

2. Object: The Instance

An **object** is a real-world instance of a class — something that actually exists in memory and can store data and perform operations.

Using our earlier bakery example:

- The `Customer` class is the blueprint.
- `customer1 = Customer("Anjali", "9876543210")` is an object — an actual customer with real details.

You can create multiple objects from a single class, each holding its own data.

3. Example: Defining and Using a Class

Let's define a `Customer` class with attributes and methods.

```
class Customer:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone

    def greet(self):
        print(f"Welcome, {self.name}!")

# Creating objects
customer1 = Customer("Anjali", "9876543210")
customer2 = Customer("Ravi", "9988776655")

# Using object methods
customer1.greet()
customer2.greet()
```

Output:

```
Welcome, Anjali!
Welcome, Ravi!
```

Here:

- `Customer` is the class
- `customer1` and `customer2` are objects of that class
- `name` and `phone` are instance variables
- `greet()` is a method (behavior)

4. Visual Understanding

Concept	Role
Class	Blueprint or template (like a form or a recipe)
Object	Actual instance with real data (filled-in form or product)
Attribute	Data associated with an object (e.g., name, phone)
Method	Function inside a class that defines object behavior

5. Why This Separation Matters

- The class allows you to **define the structure once**
- The object lets you **reuse that structure** for multiple real-world entities
- This results in **cleaner, reusable, and more organized code**

6. Practice Task

Try defining a `Book` class with the following:

- Attributes: title, author, price
- Method: `display_details()` to print the book's information

Then create two book objects and call the method on both.

The `__init__` Method and Instance Variables

1. What is the `__init__` Method?

In Python, the `__init__` method is a **special method** that is automatically called **when a new object is created** from a class.

Its primary role is to **initialize the object** — in other words, set up the object with initial data (also called attributes or instance variables).

You can think of it as a **constructor** in other programming languages.

Syntax:

```
class ClassName:
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2
```

- `self` refers to the current object being created
- The parameters passed to `__init__` are assigned to the object's attributes

2. What are Instance Variables?

Instance variables are variables that belong to a specific object (or instance) of a class. They are created inside the `__init__` method using `self`.

Each object gets its own **copy** of instance variables.

3. Example: Using `__init__` and Instance Variables

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
```

```
        self.department = department
        self.salary = salary

    def show_info(self):
        print(f"Employee: {self.name}, Department: {self.department},
Salary: ₹{self.salary}")

# Creating objects
emp1 = Employee("Aarav", "HR", 35000)
emp2 = Employee("Neha", "IT", 55000)

emp1.show_info()
emp2.show_info()
```

Output:

```
Employee: Aarav, Department: HR, Salary: ₹35000
Employee: Neha, Department: IT, Salary: ₹55000
```

Here:

- `__init__` assigns the values when the object is created
- Each object (`emp1`, `emp2`) holds its own data independently

4. Visual Summary

Element	Meaning
__init__()	Constructor method called automatically at object creation
self.variable	Instance variable unique to the object
self	Refers to the current instance of the class

5. Common Mistakes to Avoid

- Forgetting to include `self` as the first parameter in `__init__`

- Not using `self.variable` for assigning instance attributes
- Mixing up class variables (shared by all) and instance variables (unique per object) — we'll cover this distinction later

Understanding Methods: Instance Methods, Class Methods, and Static Methods

In Python classes, **methods** are functions defined inside a class. They define the **behavior** of objects and help you interact with object data.

Python provides **three types of methods** inside a class:

1. Instance Methods
2. Class Methods
3. Static Methods

Each type has a different purpose and behaves differently based on how it accesses data.

1. Instance Methods

These are the most commonly used methods. They **operate on individual object data** (instance variables) and take **`self`** as the first parameter.

Example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display_info(self):  # Instance method
        print(f"Name: {self.name}, Marks: {self.marks}")

# Usage
s1 = Student("Riya", 87)
```

```
s1.display_info()
```

Output:

```
Name: Riya, Marks: 87
```

- **`self`** refers to the object calling the method
- Instance methods can access and modify the object's data

2. Class Methods

Class methods operate on the **class itself** rather than individual objects. They take **`cls`** as the first parameter and are declared using the **`@classmethod`** decorator.

Use class methods when you need to:

- Work with **class-level data**
- Provide **alternate constructors** for creating objects

Example:

```
class Product:
    company = "Innomatics Research Labs"  # Class variable

    def __init__(self, name, price):
        self.name = name
        self.price = price

    @classmethod
    def change_company(cls, new_name):
        cls.company = new_name

# Usage
Product.change_company("IRL Pvt Ltd")
print(Product.company)
```

Output

```
IRL Pvt Ltd
```

- **`cls`** refers to the class itself
- Useful for modifying class variables shared across all objects

3. Static Methods

Static methods **do not take `self` or `cls`** as the first argument. They behave like **regular functions**, but they belong to the class namespace and are logically grouped with the class.

Use static methods for:

- Utility/helper functions
- Operations related to the class but not dependent on instance or class data

They are declared using the **`@staticmethod`** decorator.

Example:

```
class Calculator:
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def greet():
        print("Welcome to the Calculator App")

# Usage
print(Calculator.add(5, 3))
Calculator.greet()
```

Output:

```
8
Welcome to the Calculator App
```

Method Type	Decorator	First Argument	Can Access Instance Data	Can Access Class Data	Use Case
Instance Method	No decorator	self	Yes	Yes (via self)	Working with object-specific data

Class Method	@classmethod	cls	No	Yes	Working with class variables or settings
Static Method	@staticmethod	None	No	No	Utility/helper methods

Practice Task

Define a **class `BankAccount`** with:

- Instance method: **`deposit()`** to add money to the balance
- Class method: **`set_bank_name()`** to change the name of the bank
- Static method: **`bank_policy()`** to print general banking rules

Then create an account object, perform a deposit, change bank name, and call the policy method.

Encapsulation

1. What is Encapsulation?

Encapsulation is the concept of **hiding internal details** of an object and restricting direct access to them. It allows a class to **protect its data** by controlling how it's accessed or modified.

In simple terms:

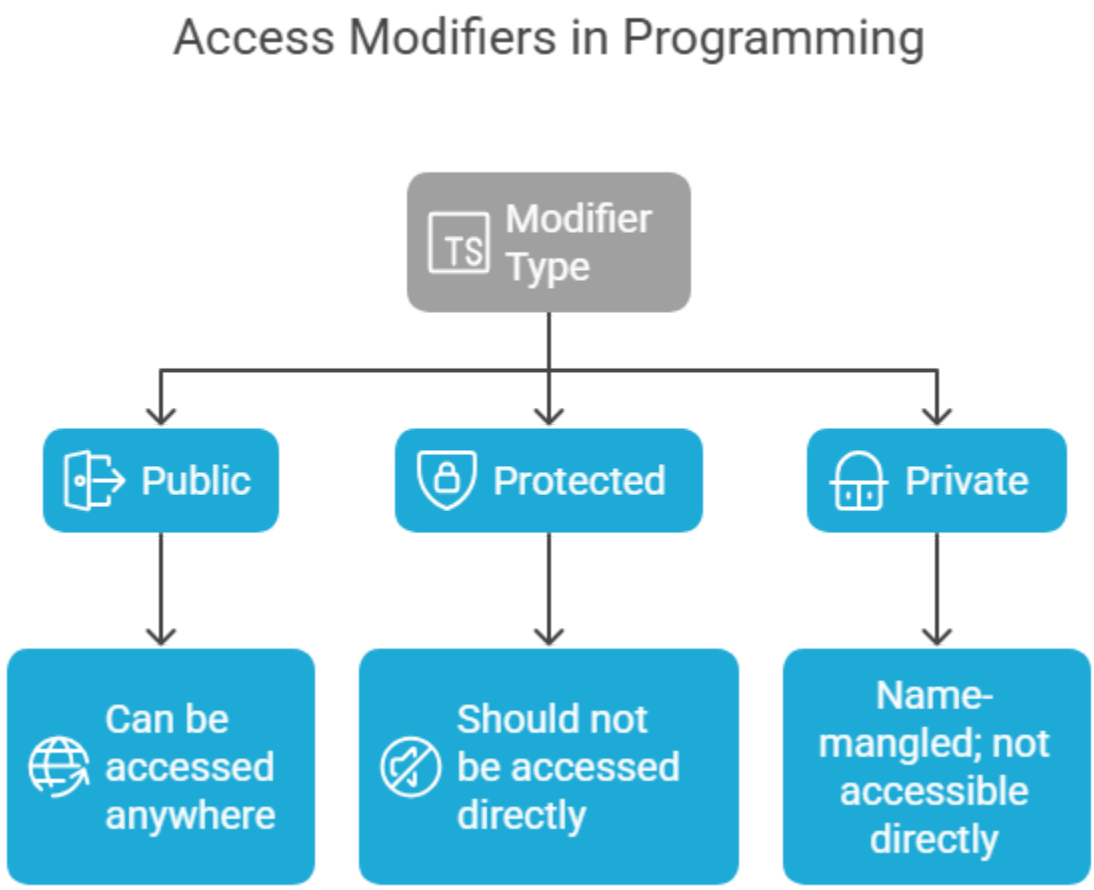
- Encapsulation means **wrapping data and related behavior inside a single unit** (class).
- It allows you to **hide implementation details** and expose only what is necessary.

This helps in:

- Avoiding accidental changes to sensitive data
- Providing a **controlled interface** for data access
- Enforcing constraints, validation, or logging while changing values

2. Encapsulation in Python

Python doesn't enforce strict access control like some other languages (e.g., Java or C++), but it **follows conventions** for access modifiers:



3. Example: Public, Protected, and Private Variables

```
class Employee:
    def __init__(self, name, salary):
        self.name = name              # public
        self._department = "HR"       # protected
        self.__salary = salary        # private(__salary)

    def show_details(self):
        print(f"Name: {self.name}, Department: {self._department}, Salary:
{self.__salary}")

emp1 = Employee("Meera", 45000)
emp1.show_details()
```

```
print(emp1.name)          # Public: Accessible
print(emp1._department)   # Protected: Should avoid direct access
# print(emp1.__salary)    # Private: Error

# Correct way to access private variable (not recommended)
print(emp1._Employee__salary)  # Name mangling
```

Output

```
Name: Meera, Department: HR, Salary: 45000
Meera
HR
45000
```

4. Getter and Setter Methods

To access or modify private variables safely, we use **getter** and **setter** methods.

Example:

```
class Account:
    def __init__(self, balance):
        self.__balance = balance  # private

    def get_balance(self):       # Getter
        return self.__balance

    def deposit(self, amount):   # Setter
        if amount > 0:
            self.__balance += amount

acc = Account(1000)
print("Initial Balance:", acc.get_balance())
acc.deposit(500)
print("Updated Balance:", acc.get_balance())
```

Output

```
Initial Balance: 1000
```

```
Updated Balance: 1500
```

- The internal balance is hidden
- Only allowed changes go through the setter (**`deposit`**)
- Prevents accidental or malicious modifications

5. Why Use Encapsulation?

- **Security**: Protects internal object state
- **Data Integrity**: Enforces validation or business rules
- **Simplicity**: Provides a clear interface to interact with the object
- **Maintainability**: Changes to internal implementation don't affect external usage

Inheritance

1. What is Inheritance?

Inheritance is a feature of Object-Oriented Programming that allows a class (called the **child** or **subclass**) to **inherit** properties and behaviors (attributes and methods) from another class (called the **parent** or **superclass**). It promotes **code reuse**, logical hierarchy, and reduces redundancy.

When one class inherits another:

- It gains access to the parent class's methods and variables.
- It can override or extend functionality if needed.

2. Why Use Inheritance?

Here's why inheritance is useful:

- **Avoids Code Duplication**: Common functionality is written once in a base class and reused across subclasses.
- **Enables Extensibility**: New features can be added without modifying existing code.
- **Establishes Relationships**: Reflects real-world hierarchical relationships (e.g., Animal → Dog, Cat).
- **Improves Maintainability**: Common changes can be made in the base class without touching all derived classes.

3. Example: Inheritance in Action

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):  # Dog inherits from Animal
    def bark(self):
        print(f"{self.name} barks.")

# Usage
d = Dog("Tommy")
d.speak()  # Inherited method
d.bark()   # Child class method
```

Here:

- **`Dog`** is the child class.
- **`Animal`** is the parent class.
- **`Dog`** can use both its own methods and those inherited from **`Animal`**.

4. Types of Inheritance in Python

Python supports five types of inheritance:

- **Single Inheritance**
- **Multiple Inheritance**
- **Multilevel Inheritance**
- **Hierarchical Inheritance**
- **Hybrid Inheritance**

Single Inheritance

Single Inheritance is a type of inheritance where a **child class inherits** from a **single parent class**. The child class gains access to all public and protected properties and methods of the parent class.

This is the **most basic form** of inheritance and is used when there is a **one-to-one relationship** between two classes.

Use Case

Single inheritance is useful when:

- You want to **extend the functionality** of an existing class without modifying it
- You need to create a **specialized version** of a general class
- You want to implement **code reuse** where the base class handles common behavior

Example: Vehicle and Car

Let's say we are building a transport-related application.
We start by creating a **general class** called **Vehicle** which contains common features like **start** and **stop**.

Later, we create a **specialized class** called **Car** that inherits from **Vehicle** and adds its own specific behavior.

```
# Parent class
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def start(self):
        print(f"{self.brand} vehicle has started.")
```

```
    def stop(self):
        print(f"{self.brand} vehicle has stopped.")

# Child class
class Car(Vehicle):  # Single inheritance
    def drive(self):
        print(f"{self.brand} car is now being driven.")

# Creating object of child class
my_car = Car("Toyota")

# Accessing parent class methods
my_car.start()     # Inherited from Vehicle
my_car.stop()      # Inherited from Vehicle

# Accessing child class method
my_car.drive()     # Defined in Car
```

Explanation:

- **`Vehicle`** is the **parent class** with methods **`start()`** and **`stop()`**.
- **`Car`** is the **child class** that inherits from **`Vehicle`** using **`Car(Vehicle)`**.
- The **`Car`** class defines an additional method **`drive()`**.
- An object of **`Car`** has access to:
    - Its own method **`drive()`**
    - Inherited methods **`start()`** and **`stop()`** from **`Vehicle`**

This demonstrates **Single Inheritance** where one child class extends a single parent class's functionality.

Multiple Inheritance

Multiple Inheritance is a type of inheritance where a single child class inherits from **two or more parent classes**.
This allows the child class to **combine behaviors and features** from multiple sources.

Use Case

Use multiple inheritance when:

- A class must inherit from **multiple specialized behaviors**
- You want to model complex hierarchies where different subclasses inherit the same base but extend in different ways
- For example, a `Suv` can combine capabilities of both `CityRide` and `OffRide`, which are both types of `Car`, which itself is a `Vehicle`

Example: Vehicle → Car → CityRide / OffRide → Suv

```
# Base class
class Vehicle:
    def start(self):
        print("Vehicle has started.")

    def stop(self):
        print("Vehicle has stopped.")

# First-level derived class
class Car(Vehicle):
    def drive(self):
        print("Driving the car.")

# Second-level derived classes
class CityRide(Car):
    def eco_mode(self):
        print("Eco mode enabled for city driving.")

class OffRide(Car):
    def hill_assist(self):
        print("Hill assist active for off-road.")

# Multiple inheritance from CityRide and OffRide
class Suv(CityRide, OffRide):
    def suv_mode(self):
        print("SUV is in 4x4 mode.")
```

```
# Create object of Suv
my_suv = Suv()

# Accessing methods
my_suv.start()          # From Vehicle
my_suv.drive()          # From Car
my_suv.eco_mode()       # From CityRide
my_suv.hill_assist()    # From OffRide
my_suv.suv_mode()       # Own method
```

Explanation

- `Vehicle` provides general methods like `start()` and `stop()`
- `Car` inherits from `Vehicle` and adds driving functionality
- `CityRide` and `OffRide` are specialized car types
  - `CityRide` adds eco-friendly features
  - `OffRide` adds off-road capabilities
- Suv inherits from **both** `CityRide` and `OffRide`
  - It gets features from all its ancestors: `Vehicle`, `Car`, `CityRide`, and `OffRide`
  - Adds its own method `suv_mode()`

Multilevel Inheritance

Multilevel Inheritance is a type of inheritance where a class inherits from a child class, which in turn inherits from another parent class. In this way, a **chain of inheritance** is formed. This allows the last class in the chain to **access all properties and behaviors** from the entire hierarchy above it.

Use Case

Multilevel inheritance is useful when:

- You want to **build upon previously defined classes step-by-step**
- Each subclass becomes a **more specialized version** of its parent
- You need to implement layered roles (e.g., `Vehicle → Car → ElectricCar`)

Example: Vehicle → Car → ElectricCar

Let's consider a real-world example:

- `Vehicle`: A base class with generic transportation behaviors
- `Car`: Inherits from `Vehicle` and adds basic car-specific behavior
- `ElectricCar`: Inherits from `Car` and adds electric-specific features like charging

```
# Base class
class Vehicle:
    def start(self):
        print("Vehicle has started.")

    def stop(self):
        print("Vehicle has stopped.")

# First-level derived class
class Car(Vehicle):
    def drive(self):
        print("Car is being driven.")

# Second-level derived class
class ElectricCar(Car):
    def charge(self):
        print("Electric car is charging.")

# Creating object of the most derived class
tesla = ElectricCar()

# Accessing methods from all levels
tesla.start()     # Inherited from Vehicle
tesla.drive()     # Inherited from Car
tesla.charge()    # Defined in ElectricCar
```

Explanation

- `Vehicle` is the top-level parent class with basic methods
- `Car` extends `Vehicle` by adding a driving feature
- `ElectricCar` extends `Car` by adding electric-specific features
- The object `tesla` of class `ElectricCar` has access to:
  - `start()` and `stop()` from `Vehicle`
  - `drive()` from `Car`
  - `charge()` from `ElectricCar`

Hierarchical Inheritance

Hierarchical Inheritance is a type of inheritance where **multiple child classes inherit from a single parent class**. Each child class gets access to the methods and attributes of the same base class but can also define its own specific behaviors.

Use Case

Hierarchical inheritance is useful when:

- You want to define a **common base class** that multiple subclasses can reuse
- Each subclass represents a **different category or type** of the same base concept
- For example, all vehicles (car, bike, truck) share a base structure but behave differently

Example: Vehicle → Car / Bike / Truck

Let's model the `Vehicle` class as a parent, and derive multiple specific vehicle types from it.

```
# Parent class
class Vehicle:
```

```python
    def start(self):
        print("Vehicle has started.")

    def stop(self):
        print("Vehicle has stopped.")

# Child class 1
class Car(Vehicle):
    def open_trunk(self):
        print("Car trunk opened.")

# Child class 2
class Bike(Vehicle):
    def kick_start(self):
        print("Bike started using kick.")

# Child class 3
class Truck(Vehicle):
    def load_cargo(self):
        print("Truck is loading cargo.")

# Creating objects
my_car = Car()
my_bike = Bike()
my_truck = Truck()

# Accessing shared methods (from Vehicle)
my_car.start()
my_bike.start()
my_truck.start()

# Accessing class-specific methods
my_car.open_trunk()
my_bike.kick_start()
my_truck.load_cargo()
```

Explanation

- `Vehicle` is the **common parent class** with shared functionality

- `Car`, `Bike`, and `Truck` are **child classes** that inherit from `Vehicle`
- Each subclass adds **specific behavior**:
  - Car → `open_trunk()`
  - Bike → `kick_start()`
  - Truck → `load_cargo()`

Each object (`my_car`, `my_bike`, `my_truck`) has access to:

- Inherited methods like `start()` and `stop()`
- Their own additional features

Benefits of Hierarchical Inheritance

- **Code reuse** through a common base class
- **Separation of concerns** — each subclass handles its own behavior
- Easy to **extend functionality** to new child types (e.g., add `Bus` later)

Hybrid Inheritance

Hybrid Inheritance is a combination of **two or more types of inheritance** (such as single, multiple, multilevel, or hierarchical) in a single program. It reflects **complex real-world relationships** where a class may have multiple sources and pass through several inheritance levels. Python supports hybrid inheritance and resolves ambiguity through the **Method Resolution Order (MRO)**.

Use Case

Use Hybrid Inheritance when:

- You need to **model a complex class hierarchy** combining behaviors from different paths
- The system involves **multiple dimensions** of classification (e.g., type of vehicle + usage purpose)

- You want to demonstrate how Python handles **MRO conflicts**

Example: Vehicle + Car → CityRide / OffRide → Suv

This is a **hybrid of multilevel and multiple inheritance**.

```
# Base class
class Vehicle:
    def start(self):
        print("Vehicle started.")

# First-level derived class
class Car(Vehicle):
    def drive(self):
        print("Car is being driven.")

# Second-level subclasses
class CityRide(Car):
    def eco_mode(self):
        print("Eco mode enabled for city driving.")

class OffRide(Car):
    def hill_assist(self):
        print("Hill assist enabled for off-road.")

# Final subclass inheriting from both CityRide and OffRide (Hybrid
Inheritance)
class Suv(CityRide, OffRide):
    def suv_mode(self):
        print("SUV 4x4 mode activated.")

# Create object of Suv
my_suv = Suv()

# Access all methods
my_suv.start()         # Inherited from Vehicle
my_suv.drive()         # Inherited from Car
my_suv.eco_mode()      # From CityRide
my_suv.hill_assist()   # From OffRide
my_suv.suv_mode()      # Own method
```

Explanation

- `Vehicle` is the base class with general vehicle behavior
- `Car` inherits from `Vehicle`
- `CityRide` and `OffRide` inherit from `Car`, each adding specialized functionality
- Suv inherits from **both `CityRide` and `OffRide`**, combining all the behavior

This example clearly shows:

- **Multilevel inheritance**: Vehicle → Car → CityRide / OffRide
- **Multiple inheritance**: Suv inherits from both CityRide and OffRide

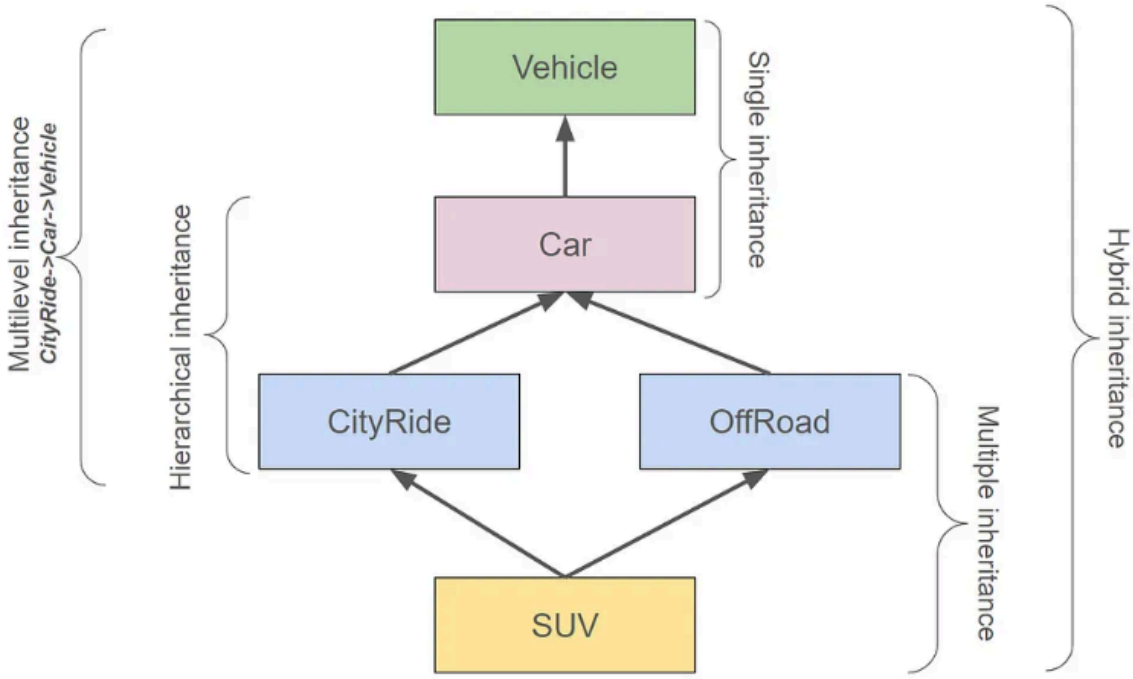
Thus, it forms a **Hybrid Inheritance** structure.

Benefits of Hybrid Inheritance

- Allows you to model **complex behaviors**
- Promotes **modular, reusable design**
- Enables building flexible and layered hierarchies

Summary

The bellow image reference will cover all types of inheritance in Python, demonstrating how they can be used together to form complex inheritance structures.



example of different type of Inheritance

Method Overriding in Python

What is Method Overriding?

Method overriding occurs when a **child class provides its own version** of a method that already exists in its parent class. The child's method **replaces** the parent's method when called from the child object.

Why Override Methods in Child Classes?

- To **customize or extend** the behavior of the inherited method
- To implement **specific logic** for the child class while reusing the general interface
- To support **polymorphism** where different classes respond differently to the same method call

Basic Example: Method Overriding

```
class Vehicle:
    def start(self):
        print("Vehicle has started.")

class Car(Vehicle):
    def start(self):  # Overriding the parent's start method
        print("Car has started with keyless ignition.")

my_car = Car()
my_car.start()  # Output: Car has started with keyless ignition.
```

In the above code, the `start()` method in `Car` overrides the same method from `Vehicle`.

Using `super()` to Call Parent Method

Sometimes, we want to override a method **but still use** the parent's version as part of the logic. This is done using `super()`.

Example with `super()`:

```
class Vehicle:
    def start(self):
        print("Vehicle started.")

class Car(Vehicle):
    def start(self):
        super().start()  # Call parent method
        print("Car engine check complete. Ready to drive.")

my_car = Car()
my_car.start()
```

Output:

```
Vehicle started.
Car engine check complete. Ready to drive.
```

MRO Behavior in Multiple Inheritance

In multiple inheritance, Python resolves method calls using **MRO (Method Resolution Order)**.

The method is resolved by:

1. Looking at the class itself
2. Then scanning parents **from left to right** in the class definition

```
class A:
    def show(self):
        print("From A")

class B:
    def show(self):
        print("From B")

class C(A, B):
    pass

c = C()
c.show()  # Output: From A
print(C.__mro__)  # Shows order: C → A → B → object
```

Constructor Overriding in Python

What is Constructor Overriding?

In Python, the constructor method is `__init__()`. **Constructor overriding** happens when the child class defines its **own `__init__()`**, replacing the parent's constructor.

How It Affects Inheritance

When the child class has its own `__init__()` method:

- The parent's `__init__()` is **not automatically called**
- If you want to run both constructors, you must **explicitly call the parent's `__init__()`** using `super()`

Example Without `super()`

```
class Vehicle:
    def __init__(self):
        print("Vehicle constructor")

class Car(Vehicle):
    def __init__(self):
        print("Car constructor")

my_car = Car()
```

Output:

```
Car constructor
```

Here, the `Vehicle` constructor is skipped because it's not explicitly called.

Calling Parent Constructor Using `super()`

```
class Vehicle:
    def __init__(self):
        print("Vehicle constructor")

class Car(Vehicle):
    def __init__(self):
        super().__init__()  # Calls parent constructor
        print("Car constructor")

my_car = Car()
```

Output:

```
Vehicle constructor
Car constructor
```

Why Use Constructor Overriding

- To initialize **additional attributes** specific to the child class
- To perform **additional setup logic** without losing the parent's constructor behavior

Polymorphism in Python

What is Polymorphism?

- Polymorphism means **many forms**.
- The **same method name** behaves **differently** for different classes.

Example:

- A **Car** and a **Bike** both can `start()`, but the way they start is different.

Why Use Polymorphism?

- **Flexibility**: Same method name, different behavior.
- **Cleaner code**: No need for too many conditions.
- **Easier to extend**: New vehicle types can be added easily.

Method Overriding (Real Polymorphism in Python)

- A **child class** provides its **own version** of a method from the **parent class**.

Example:

```
class Vehicle:
    def start(self):
        print("Vehicle is starting...")

class Car(Vehicle):
    def start(self):
        print("Car engine starting with a key...")

class Bike(Vehicle):
    def start(self):
        print("Bike starting with a button...")

# Polymorphism Example
```

```
for vehicle in (Car(), Bike()):
    vehicle.start()
```

Output:

```
Car engine starting with a key...
Bike starting with a button...
```

Polymorphism with Functions

- A function can work with **different vehicle objects**.

Example:

```
def begin_journey(vehicle):
    vehicle.start()

begin_journey(Car())
begin_journey(Bike())
```

Output:

```
Car engine starting with a key...
Bike starting with a button...
```

Important Points

- Python **supports polymorphism naturally** through method overriding.
- True **method overloading** (same method name with different parameters) is **not supported directly** like in Java.

Abstraction

What is Abstraction?

- **Abstraction** means **hiding the internal implementation** and **showing only important details**.
- It helps to **reduce complexity** and **improve code focus**.

Example:
When you drive a **Car**, you just press the **start button** —
you don't need to know how the engine works internally.

Why Use Abstraction?

- **Hides complexity** from the user.
- **Focuses** only on the necessary features.
- **Makes code more secure and cleaner**.
- **Improves maintainability** and **understanding**.

How Abstraction Works in Python?

- Python uses the **abc module** (**Abstract Base Classes**) to implement abstraction.
- An **Abstract Class**:
  - Cannot be instantiated directly.
  - Contains **abstract methods** (methods without implementation).
- A **Child Class**:
  - Inherits from the abstract class.
  - Must **implement all abstract methods**.

Syntax:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):  # Abstract Class
```

```
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):  # Concrete Class
    def start(self):
        print("Car is starting with a button...")

my_car = Car()
my_car.start()
```

Output:

```
Car is starting with a button...
```

Key Points:

Concept	Explanation
Abstract Class	Created using ABC
Abstract Method	Defined using `@abstractmethod`
Cannot Create	Object of abstract class directly
Must Implement	Child class must define abstract methods